DFSha

Sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad

Informe técnico — Proyecto 1

SI3007 Sistemas Distribuidos / ST0263 Tópicos Especiales en Telemática

Universidad EAFIT — 2026-2

Opción de arquitectura asignada: 1 (C/S con composición y distribución S2S)

Repositorio: github.com/DrotsGS/Topicos-de-Telematica

1. Objetivo y marco teórico

Objetivo. Diseñar e implementar un sistema de archivos distribuido (DFS) que almacene archivos grandes repartidos en varios nodos, con alta disponibilidad, rendimiento, transparencia y seguridad, en el que tanto la escritura como la lectura se distribuyan entre los nodos.

Marco teórico. Un DFS permite compartir y acceder de forma concurrente a archivos guardados en distintos nodos. Hay dos familias. Los DFS basados en bloques (NFS, AFS, SMB) reparten los bloques de un archivo entre nodos y suelen integrarse al sistema operativo, de modo que la API es la misma para archivos locales y remotos. Los basados en objetos (AWS S3) tratan el archivo como unidad: se lee o se escribe completo, no admiten actualización parcial y siguen un enfoque WORM (write once, read many). El cliente usa un SDK o CLI propio, y por dentro el sistema suele particionar en bloques para escalar y tolerar fallas.

DFSha toma de ambos. Como GFS y HDFS (las referencias del enunciado), es un DFS por bloques con un nodo de metadatos (NameNode) y nodos de datos (DataNodes), y el cliente parte y reensambla los archivos. Como los sistemas de objetos, la semántica es WORM y el acceso es por CLI y SDK, no montado en el sistema operativo. Para la alta disponibilidad del nodo de metadatos se usa Raft, un algoritmo de consenso que mantiene un log replicado con un solo orden de operaciones mientras la mayoría de los nodos esté viva.

2. Descripción del servicio y problema abordado

El problema: guardar archivos más grandes que un disco, a más velocidad que una sola máquina, sin perderlos cuando se cae una máquina y sin que nadie más pueda leerlos.

El servicio ofrece a cada usuario un árbol jerárquico tipo Linux. Cada archivo se parte en bloques de 128 MB; cada bloque se guarda cifrado en tres DataNodes distintos; los metadatos viven en tres NameNodes replicados con Raft.

2.1 Requisitos funcionales

RF	Requisito	Implementación
RF1	Gestión: ls, cd, mkdir, rmdir, rm…	`ls`, `cd`, `pwd`, `mkdir`, `rmdir`, `rm`, `stat`, `chmod`, con rutas relativas
RF2	Transferencia: send(), receive()	`put` y `get` en la CLI; `send()` y `receive()` en el SDK
RF3	Acceso: open, close, read, write, lock	SDK: `open(path, 'r'|'w')`, `read`, `seek`, `tell`, `write`, `close`, `lock`. `send`/`receive` están implementados sobre estas primitivas

Bajo WORM, `open(path, "w")` toma el lock de escritor único (un lease); `write()` agrega al final bloque a bloque sin conocer el tamaño final, y `close()` hace el commit. La lectura es de acceso aleatorio: `seek()` y `read()` bajan solo el bloque que contiene el offset.

2.2 Requisitos no funcionales

RNF	Cómo se cumple
RNF1 Escalabilidad	Los datos no pasan por el NameNode: el cliente habla directo con los DataNodes. Hash consistente: agregar un nodo mueve ~1/N de los bloques (medido: 17,4 %).

RNF2 Alta disponibilidad	Datos: 3 réplicas y re-replicación automática. Control: 3 NameNodes con Raft; un líder nuevo atiende en 1–3 s.
RNF3 Consistencia	Metadatos por log de Raft y lectura solo del líder. Datos inmutables (WORM) con sha256 verificado; un archivo o no existe o existe completo.
RNF4 Particionamiento	Bloques de 128 MB repartidos por hash consistente con 150 nodos virtuales.
RNF5 Rendimiento	Bloques de un archivo en paralelo; servidores con pool de hilos; múltiples clientes concurrentes.
RNF6 Seguridad	AES-256-GCM en el cliente; mTLS en todos los pares; usuario y contraseña con tokens HMAC; dueño, grupo y permisos; tokens de bloque firmados; Raft cifrado.
RNF7 Transparencia	Localización total (ni DataNodes ni líder visibles al usuario); acceso relajado a CLI y SDK, nunca estático.

3. Arquitectura del sistema

Opción 1: cliente/servidor, con el servidor implementado por composición y distribución (S2S) dentro de una red privada, en un esquema maestro–trabajador. Se descartó la opción 2 (P2P con SuperPeers) porque el diseño tiene una sola autoridad de metadatos, replicada pero única, y no requiere metadatos repartidos por DHT.

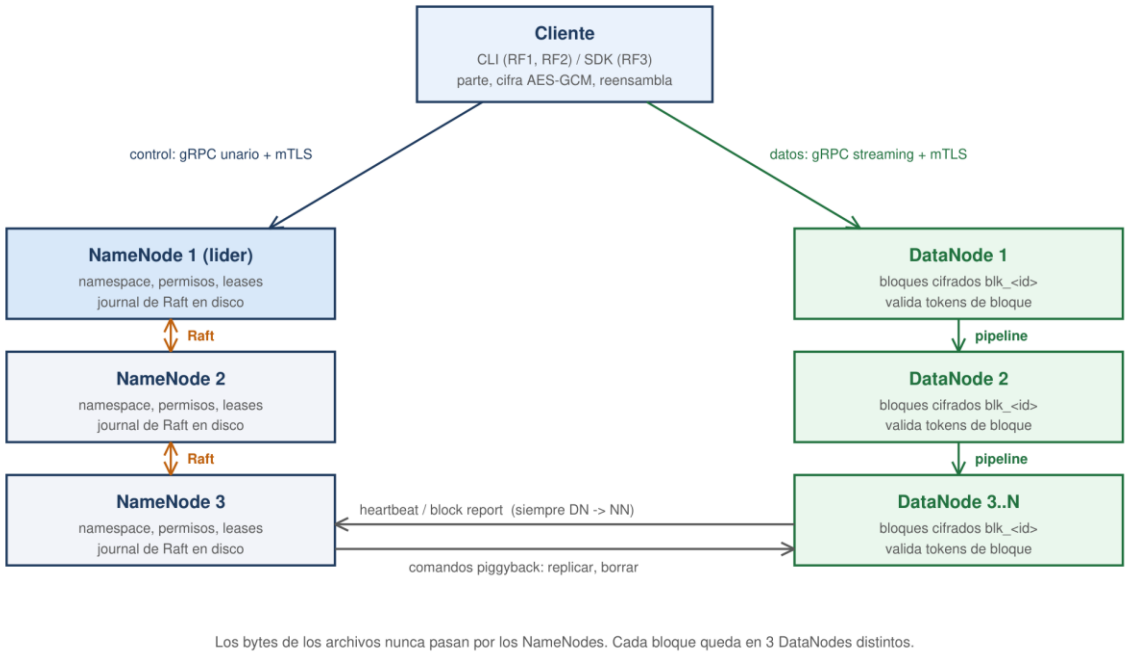


Figura 1. Componentes y planos de comunicación.

Componente	Responsabilidad	Código
Cliente	Parte y reensambla, cifra, encuentra al líder, habla con los DataNodes	`src/client/cli.py`, `src/client/sdk.py`
NameNode (ControlNode)	Namespace, permisos, leases, colocación, re-replicación, recolector de bloques	`src/namenode/server.py`, `estado.py`, `namespace.py`

DataNode	Guarda bloques, valida tokens, reenvía por el pipeline, obedece comandos	`src/datanode/server.py`

3.1 Las cinco costuras

Todo lo que se sabía que iba a cambiar quedó detrás de una interfaz en `src/common/interfaces.py`, con una versión deliberadamente simple al inicio. Las cinco evolucionaron sin tocar el resto del sistema:

Interfaz	v1 (semana 6)	Versión final
BlockPlacer	round robin	hash consistente, 150 nodos virtuales
MetadataStore	en memoria	Raft (pysyncobj) con journal en disco
Replicator	copia única	pipeline DataNode → DataNode
AuthProvider	todo pasa	usuarios y grupos, tokens HMAC, tokens de bloque
BlockCipher	passthrough	AES-256-GCM en el cliente

4. Especificación de protocolos y APIs

Todas las comunicaciones usan gRPC sobre HTTP/2 con Protocol Buffers, salvo Raft. Se eligió gRPC frente a REST porque el plano de datos necesita streaming con control de flujo para bloques de 128 MB, porque Protocol Buffers transporta bytes sin base64 y porque el contrato tipado genera los stubs de los tres programas. El contrato está en cuatro archivos `.proto`: uno de tipos comunes y uno por par de comunicación. La especificación detallada, con cada mensaje y su semántica, está en `docs/protocolos.md`.

Par	Protocolo	Inicia	Contenido
Cliente ↔ ControlNode	gRPC unario (`NameNodeService`)	cliente	Login, Ping, Ls, Mkdir, Rmdir, Rm, Stat, Chmod, Create, AddBlock, Renew, Complete, Abort, Open
Cliente ↔ DataNode	gRPC streaming (`DataNodeService`)	cliente	PutBlock (client streaming), GetBlock (server streaming)
ControlNode ↔ ControlNode	Raft (pysyncobj) sobre TCP cifrado	líder / candidato	operaciones del namespace en un log replicado
ControlNode ↔ DataNode	gRPC unario (`ControlService`)	siempre el DataNode	Heartbeat, BlockReport, BlockReceived; comandos piggyback en la respuesta
DataNode ↔ DataNode	gRPC streaming (`PutBlock` con pipeline, `ReplicateTo`)	el DataNode con el bloque	replicación y re-replicación

4.1 Decisiones de protocolo no obvias

- **Piggyback:** el NameNode nunca abre una conexión hacia un DataNode. Los comandos (replicar, borrar) viajan en la respuesta al heartbeat. Resuelve NAT y firewalls, y agregar un DataNode no requiere tocar el NameNode.

- **Latido a todos:** cada DataNode late y reporta a los tres NameNodes, en un hilo por NameNode, para que un líder recién elegido ya sepa qué nodos están vivos y qué bloques tienen.
- **Solo el líder atiende:** un seguidor responde UNAVAILABLE y el cliente busca al líder con Ping y reintenta. Leer solo del líder evita devolver un namespace atrasado.
- **Errores tipados:** cada error del dominio tiene su StatusCode (UNAUTHENTICATED, PERMISSION_DENIED, NOT_FOUND, ALREADY_EXISTS, FAILED_PRECONDITION, INVALID_ARGUMENT, UNAVAILABLE), definidos en una sola tabla del NameNode.

5. Algoritmos de particionamiento y distribución

- **Partición.** el NameNode corta el tamaño en bloques de 128 MB con un contador explícito (el último es parcial). Con `AddBlock` el cliente pide bloques uno a uno sin conocer el total.
- **Colocación.** `ConsistentHashPlacer` ubica cada `block_id` en un anillo con 150 nodos virtuales por DataNode vivo y toma los tres primeros nodos físicos distintos en sentido horario. Con 10 000 claves, pasar de 4 a 5 nodos movió el 17,4 % (con módulo sería ~80 %) y el reparto entre 4 nodos quedó en 22,1 / 27,0 / 26,2 / 24,7 %. El hash es política de colocación inicial, no mecanismo de búsqueda: el NameNode recuerda dónde quedó cada bloque.
- **Escritura por pipeline.** el cliente sube cada bloque una vez al primer DataNode, que lo guarda y lo reenvía al segundo, y este al tercero. Si un nodo de la cadena falla se salta: se pierde una copia, no la escritura.
- **Lectura con failover.** los bloques se bajan en paralelo; cada uno de cualquier réplica viva, saltando a la siguiente si una falla o su sha256 no coincide.
- **Re-replicación y recolector.** cada 5 s el líder busca bloques con menos de 3 copias vivas y le pide a un DataNode que tenga una que la copie a los que faltan. Los bloques sin dueño (archivo borrado, lease vencido o detectados en un block report) se mandan borrar.
- **Estado blando y estado durable.** las ubicaciones reales no se persisten: se reconstruyen con los block reports, como en HDFS. Lo durable (namespace, block map, leases) va en el log de Raft.

6. Entorno de ejecución

Python 3.12 (Docker) y 3.14 (Windows), gRPC 1.83, protobuf 7.36, pysyncobj 0.3.14 y cryptography 50. Toda dirección sale de variables de entorno, así que el mismo código corre sin cambios en tres modalidades:

Modalidad	Cómo	Uso
Procesos nativos	un proceso por nodo (`python src/namenode/server.py`, `python src/datanode/server.py`), con mTLS opcional	desarrollo, demo, mediciones
Docker, una máquina	`docker-compose.yml`: 3 NameNodes con Raft, 4 DataNodes y un cliente, con mTLS	probar la imagen y la red
Una VM por nodo (AWS Academy)	`deploy/docker-compose-namenode.yml` y `-datanode.yml`, una por rol, con las IPs privadas en `.env`	la entrega

El despliegue por rol sigue el taller de nginx + WordPress del curso, que usa un docker-compose por tipo de servicio con la IP privada como variable, y la generación de certificados con openssl. A diferencia del taller no se usa Let's Encrypt: lo que se autentica son nodos por IP privada y en ambos sentidos (mTLS), para lo cual sirve una CA propia (`scripts/gen_certs.py`). Los Security Groups abren 50051 (NameNode), 50052 (Raft, solo entre NameNodes) y 50060 (DataNode, para clientes y otros DataNodes). La guía paso a paso está en el README del repositorio.

7. Pruebas y análisis de resultados

7.1 Pruebas automáticas

161 pruebas con pytest, que levantan sus propios nodos en puertos efímeros y corren en unos 30 s. Cubren:

- el namespace y cada StatusCode de cada RPC;
- el DataNode aislado: streaming, tamaños, temporales, tokens válidos e inválidos;
- punta a punta: subir, borrar, bajar y comparar el hash con bloques parciales;
- el pipeline (incluido un nodo caído en medio), el recolector y la re-replicación;
- lectura con failover ante un nodo caído y ante una copia corrupta;
- leases que vencen y se renuevan, y AddBlock;
- permisos por dueño y grupo, sticky, chmod, cd y rutas relativas;
- AES-GCM (ida y vuelta, chunks movidos, clave equivocada), los DataNodes sin texto claro, y mTLS (sin certificado o de otra CA no conecta);
- el SDK: escritura por partes, acceso aleatorio, el lock, aborto ante excepción;
- tres NameNodes con Raft: un solo líder, replicación, caída del líder, sin mayoría y reinicio desde el journal.

Además se verificó la CLI real, como subprocesos, contra un clúster de 3 NameNodes y 4 DataNodes con mTLS, y la prueba de aceptación del hito 1 (500 MB, borrar, bajar y comparar el sha256).

7.2 Throughput

Medido sobre procesos reales en una sola máquina (Windows 11 / Python 3.14.6), con bloques de 16 MB, cifrado AES-GCM y mTLS. Cada fila sube un archivo, lo baja y verifica el sha256.

Escenario	Tamaño	Subida (MB/s)	Bajada (MB/s)	Hash
1 DN, r=1	10 MB	89.2	92.8	ok
1 DN, r=1	50 MB	227.0	125.0	ok
1 DN, r=1	200 MB	229.6	116.2	ok
1 DN, r=1	500 MB	214.8	96.9	ok
4 DN, r=1	10 MB	101.8	88.8	ok
4 DN, r=1	50 MB	230.5	129.3	ok
4 DN, r=1	200 MB	231.7	120.2	ok
4 DN, r=1	500 MB	219.1	111.8	ok

4 DN, r=3	10 MB	37.9	96.2	ok
4 DN, r=3	50 MB	85.5	105.0	ok
4 DN, r=3	200 MB	86.7	112.8	ok
4 DN, r=3	500 MB	82.5	109.3	ok

Análisis:

- **Replicar cuesta ~2,6× en escritura y nada en lectura:** la subida con 3 réplicas va a ~85 MB/s frente a ~220 MB/s con una: cada bloque se escribe tres veces en el mismo disco y el pipeline es store-and-forward. En máquinas separadas el costo es menor porque cada copia usa otro disco, aunque la latencia sigue sumándose por salto.
- **1 y 4 DataNodes rinden igual en una máquina:** con un solo equipo, los cuatro DataNodes comparten disco y CPU, y el cuello de botella es el cliente (cifrar AES-GCM y serializar en Python). Por eso el beneficio del paralelismo entre nodos (RNF5) no se ve aquí. Sin cifrado, en la semana 10 la bajada de 500 MB había pasado de 44,8 MB/s con un DataNode a 102,2 MB/s con cuatro. La medición que lo muestra de verdad es en AWS, con un DataNode por máquina.
- **Archivos pequeños:** con 10 MB la conexión, el login y el handshake TLS pesan proporcionalmente más.

7.3 Recuperación ante fallas

Falla	Resultado
Se mata el NameNode líder (3 NameNodes)	Un nuevo líder atiende escrituras a los 1,13 s (entre 1,1 y 3,2 s en varias corridas)
Se mata un DataNode (4 DataNodes, r=3)	Sus bloques vuelven a tener 3 copias a los 4,01 s; el nodo se declara muerto 6 s después de su último latido
Lectura tras las dos fallas	sha256 idéntico
Se reinician los 3 NameNodes	El namespace se recupera del journal (prueba automática)
Quedan 1 de 3 NameNodes	No se escribe: Raft prefiere consistencia (prueba automática)

Con la configuración de producción (`TIMEOUT_DATANODE` = 30 s) la re-replicación empieza entre 27 y 30 s después de la caída; en la medición se usó 6 s para no medir solo espera.

7.4 Errores encontrados al medir

Las pruebas con procesos reales encontraron dos errores que las pruebas en un solo proceso no veían. (1) El DataNode latía a los NameNodes en serie; en Windows una conexión a un NameNode caído tarda 2,3 s en fallar, lo que dejaba el latido real cerca del límite de 6 s. Se pasó a un hilo por NameNode. (2) La primera subida de un usuario sin clave fallaba, porque varios hilos creaban y leían el archivo de clave a la vez. Se agregó un lock y escritura atómica, con prueba de regresión.

8. Decisiones de diseño, limitaciones y trabajo futuro

Las 20 decisiones (qué se decidió, por qué, qué se descartó y a qué obliga) están en `docs/decisiones.md`. Las de mayor peso:

#	Decisión	Razón principal
D2	WORM: bloques inmutables	elimina la reconciliación entre réplicas
D3	El cliente particiona	el NameNode fuera del camino de los datos (RNF1, RNF5)
D6	`rm` gana sobre una subida en curso	un path nunca queda preso de un cliente muerto
D11/D15	Ubicaciones como estado blando	se reconstruyen con block reports; no se escribe al log en cada latido
D13	Pipeline store-and-forward	el cliente sube cada bloque una vez
D14	Raft con biblioteca; solo el líder atiende	un orden único; nunca datos atrasados
D18	Tokens de bloque validados localmente	la ACL protege también el plano de datos
D19	Cifrado en el cliente	ni un DataNode comprometido ve texto claro

Limitaciones conocidas:

- el despliegue en AWS está preparado y documentado, pero las mediciones son de una sola máquina;
- la clave de cifrado es por usuario y local: para leer desde otra máquina hay que llevarla;
- el pipeline reenvía el bloque completo y no chunk a chunk, lo que suma latencia por salto;
- no hay 2FA ni API keys; la autenticación es usuario/contraseña con tokens que expiran;
- `rmdir` no es recursivo desde el protocolo (D5).

9. Conclusiones

DFSha cumple los tres requisitos funcionales y los siete no funcionales del enunciado con una arquitectura de opción 1 de tipo maestro–trabajador. Hay tres resultados centrales. Separar el plano de control del de datos permite que los NameNodes nunca toquen los bytes. La replicación existe en los dos planos: Raft para los metadatos, pipeline y re-replicación para los bloques. Y la seguridad está en las capas donde un atacante podría estar: canal (mTLS), almacenamiento (AES-GCM en el cliente) y autorización (permisos y tokens de bloque). Las mediciones confirman la recuperación automática ante la caída de un NameNode y de un DataNode, y muestran honestamente el costo de replicar y el límite de medir en una sola máquina.

Referencias

- S. Ghemawat, H. Gobioff, S. Leung. The Google File System. SOSP 2003.
- K. Shvachko, H. Kuang, S. Radia, R. Chansler. The Hadoop Distributed File System. MSST 2010.
- D. Ongaro, J. Ousterhout. In Search of an Understandable Consensus Algorithm (Raft). USENIX ATC 2014.
- D. Karger et al. Consistent Hashing and Random Trees. STOC 1997.
- gRPC: https://grpc.io — pysyncobj: https://github.com/bakwc/PySyncObj
- Curso SI3007: enunciado del Proyecto 1 y taller docker-nginx-wordpress-ssl-letsencrypt (github.com/si3007eafit/si3007-262).